

Practical Session 6

Task 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
j_rakshitha@RAKSHITHA: ~  
Enter message: Hello from Client 1  
open FIFO: No such file or directory  
j_rakshitha@RAKSHITHA:~$ pwd  
/home/j_rakshitha  
j_rakshitha@RAKSHITHA:~$ ls -l ossp_fifo  
prw-r--r-- 1 j_rakshitha j_rakshitha 0 Sep 17 08:30 ossp_fifo  
j_rakshitha@RAKSHITHA:~$ ./fifo_client  
Practical Session 6 - Task 1: FIFO Client  
Enter message: Hello from Client 1  
Message sent successfully.  
j_rakshitha@RAKSHITHA:~$ ./fifo_client  
Practical Session 6 - Task 1: FIFO Client  
Enter message: Hello from Client 2  
Message sent successfully.  
j_rakshitha@RAKSHITHA:~$ ./fifo_client  
Practical Session 6 - Task 1: FIFO Client  
Enter message: exit  
Message sent successfully.  
j_rakshitha@RAKSHITHA:~$ |  
  
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano fifo_server.c  
j_rakshitha@RAKSHITHA:~$ nano fifo_client.c  
j_rakshitha@RAKSHITHA:~$ ^C  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g fifo_server.c -o fifo_server  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g fifo_client.c -o fifo_client  
j_rakshitha@RAKSHITHA:~$ ./fifo_server  
Practical Session 6 - Task 1: FIFO Server  
FIFO created: ossp_fifo  
Server waiting for client messages...  
Server received: Hello from Client 1  
Server response: Message processed successfully.  
Server received: Hello from Client 2  
Server response: Message processed successfully.  
Exit message received.  
Server stopped and FIFO removed.  
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 fifo_server.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO_NAME "ossp_fifo"

int main(void)
{
    char buffer[256];

    printf("Practical Session 6 - Task 1: FIFO Server\n");

    /* Create the named pipe */
    if (mkfifo(FIFO_NAME, 0666) == -1)
    {
        /* It may already exist, which is okay */
        printf("FIFO already exists or could not be newly created.\n");
    }
    else
    {
        printf("FIFO created: %s\n", FIFO_NAME);
    }

    printf("Server waiting for client messages...\n");

    while (1)
    {
        int fd = open(FIFO_NAME, O_RDONLY);

        if (fd == -1)
        {
            perror("open");
            return 1;
        }

        ssize_t bytes = read(fd, buffer, sizeof(buffer) - 1);

        close(fd);
    }
}
```

```

    if (bytes > 0)
    {
        buffer[bytes] = '\0';

        if (strcmp(buffer, "exit") == 0)
        {
            printf("Exit message received.\n");
            break;
        }

        printf("Server received: %s\n", buffer);
        printf("Server response: Message processed successfully.\n");
    }

    unlink(FIFO_NAME);

    printf("Server stopped and FIFO removed.\n");

    return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute ^{^C} Location

```
j_rakshitha@RAKSHITHA: ~  
GNU nano 8.7.1 fifo_client.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
#define FIFO_NAME "ossp_fifo"  
  
int main(void)  
{  
    char message[256];  
  
    printf("Practical Session 6 - Task 1: FIFO Client\n");  
  
    printf("Enter message: ");  
    fflush(stdout);  
  
    if (fgets(message, sizeof(message), stdin) == NULL)  
    {  
        return 1;  
    }  
  
    message[strcspn(message, "\n")] = '\0';  
  
    int fd = open(FIFO_NAME, O_WRONLY);  
  
    if (fd == -1)  
    {  
        perror("open FIFO");  
        return 1;  
    }  
  
    write(fd, message, strlen(message));  
  
    close(fd);  
  
    printf("Message sent successfully.\n");  
  
    return 0;  
}
```

Task 2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

[illegible]

j_rakshitha@RAKSHITHA: ~

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

Received SIGUSR1

User-defined signal handled successfully.

Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...

Received SIGTERM

Terminating the program safely...

j_rakshitha@RAKSHITHA:~\$ |

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 signal_handler.c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

/* Signal handler function */
void handle_signal(int signal_number)
{
    if (signal_number == SIGINT)
    {
        printf("\nReceived SIGINT (Ctrl+C)\n");
        printf("Signal handler executed asynchronously.\n");
    }
    else if (signal_number == SIGTERM)
    {
        printf("\nReceived SIGTERM\n");
        printf("Terminating the program safely...\n");
        exit(0);
    }
    else if (signal_number == SIGUSR1)
    {
        printf("\nReceived SIGUSR1\n");
        printf("User-defined signal handled successfully.\n");
    }

    fflush(stdout);
}

int main(void)
{
    printf("Practical Session 6 - Task 2: Signal Handling\n");
    printf("Process PID: %d\n", getpid());
    printf("Supported signals: SIGINT, SIGTERM, SIGUSR1\n\n");

    /* Register signal handlers */
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Waiting for signals...\n");
    printf("Press Ctrl+C for SIGINT.\n");
    printf("Use another terminal to send SIGUSR1 or SIGTERM.\n\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```